The Threads

The Threads

A personal account of how Generative Specification came together

---

An old friend from university called me one afternoon to show me something.

He had been experimenting with the latest Claude model and had built a small working program from a description alone — no boilerplate, no scaffold, just a precise statement of what he wanted and why. He wanted me to see it. He thought I would find it interesting.

He was right, but not in the way either of us expected. I did not find it interesting as a productivity tool. I found it interesting as a question: *where exactly does this break, and why?*

The Experiment

The question I started with was deliberately naive.

Can I build a complete, working system without writing a single line of code?

Not "can AI help me write code faster" — that was everyone's question. I wanted to understand whether the line between describing a system and having a machine build it could be made thin enough to disappear.

The answer was: sometimes, and the variance was the whole problem.

Some sessions produced clean, working software from a paragraph of intent. Others produced plausible-looking code that collapsed the moment it touched real data. The same model, the same session length, wildly different quality. Something in the input was the variable.

I spent months understanding that variable. CodeSeeker came out of those months — a code intelligence tool I built entirely through specification, not writing the implementation myself but writing precise descriptions of what it had to do, what constraints it had to satisfy, how its outputs would be verified. By the time it was working, I had not written a line of application code. I had written a blueprint.

That was the first realization: the blueprint was the work.

What the Blueprint Required

The more precise the description, the cleaner the output. That much was obvious quickly. What took longer to understand was that precision had a structure — that certain kinds of description produced reliably good results and others produced reliably mediocre ones regardless of how carefully they were written.

The structure that worked was one I already knew from a decade of professional software development. Domain-Driven Design gave you a shared vocabulary between the problem and the code. Test-Driven Development gave you executable specifications of behavior before any implementation existed. Clean architecture gave you clear boundaries that separated business logic from infrastructure. These disciplines had always been described as

good practice. Nobody fully applied them because the cost was too high: a human developer with limited hours had to make pragmatic choices.

The cost had changed. An AI assistant does not experience deadline pressure. It does not deprioritize invariants because the sprint ends Friday. It does not decide that this function is simple enough to skip the contract. Given a specification that encodes what correct behavior looks like, it will satisfy it — not because it understands correctness, but because it is extraordinarily good at matching precisely stated patterns.

The disciplines that had always been right but too expensive were now affordable. The blueprint — architectural constitution, decision records, use cases, behavioral contracts, quality gates — was not documentation. It was the program. The code was what the program derived.

The Imitation Game

Late in the year I came across a biographical comic about Alan Turing — *The Imitation Game*, the graphic novel adaptation of the story. I had known the broad outlines of the story for years but had not sat with the details.

Reading it, I found myself thinking about what Turing had actually shown: that computation was not fundamentally about machines. It was about formal description. A sufficiently precise description of a procedure could be executed by any system capable of following rules — a machine, a person with a pencil and paper, or something we had not yet built. The machine was not the interesting part. The description was.

That sent me back to the formal methods tradition I had learned in pieces during my career but never fully connected. Hoare's pre- and postconditions. Dijkstra's weakest precondition calculus. The Curry-Howard correspondence — the structural equivalence between programs and proofs, which meant that a type system was not just a lint tool but a way of encoding correctness as a property the compiler could verify. These ideas had always been treated as academic, too expensive for production use.

The same cost collapse that had happened to clean architecture had happened here too. Annotation fatigue — the reason developers skipped formal properties in practice — disappears when the annotator is an AI. Given a specification that encodes what invariants must hold, the AI enforces them without judgment about whether the sprint schedule permits it.

The formal tradition had been waiting for an executor that would never get tired. The executor existed now.

Loom

What the formal tradition needed was a language the AI could work with naturally — one where correctness properties were part of the grammar rather than layered on afterward, and where the structure was readable enough that a language model could extend it without disambiguation.

The answer was not to invent correctness from scratch. Rust had already done most of that work — a type system that enforces memory safety, ownership, and a substantial set of formal correctness properties at compile time, with no runtime cost. The insight was to build a transpiler that sits above Rust, adding the structural disciplines GS had accumulated — the architectural layers, the behavioral contracts, the information flow constraints — in a form that compiles down to Rust's guarantees. Fifteen formally proved properties, inherited rather than invented, plus the specification structure the methodology required.

The idea was not a new programming language in the sense of inventing new semantics. It was a new surface for an existing foundation — one designed so that a specification written for a human reader could also be compiled by a machine into formally correct code. The formal tradition stretches from Aristotle's categories through

Curry-Howard to Rust's type system. The contribution was connecting the specification layer to that foundation in a way that made AI-assisted development formally grounded rather than statistically probable.

Biological Isomorphisms

Once the first four tiers were working — once I was not writing code, not reading it, not managing infrastructure, not diagnosing bugs — a different question arrived.

What is a GS-governed system, formally?

Not what does it do, but what kind of thing is it. A system with a complete telos — from the Greek for purpose or end: the formal statement of what the system exists to do, precise enough to govern every decision made in its construction and evolution — a type system that enforces its own constraints, a harness that continuously verifies its behavior, a monitoring layer that detects and corrects drift. What has that structure?

The answer came from biology. Maturana and Varela had formalized autopoiesis — the property of a system that continuously regenerates its own components through its own processes. The immune system has memory of past threats and responds to new ones without relearning from scratch. Organisms express different behaviors from the same genome depending on context. Cells maintain their boundaries while metabolizing the environment.

These were not metaphors for what GS-governed systems do. They were structural descriptions of the same properties, arrived at by a completely different path. Boundary maintenance. Error correction before propagation. Immune memory. Adaptive response within governed constraints. The formal mappings — Biological Isomorphisms — documented the correspondence precisely, not as analogy but as isomorphism: the same formal structure instantiated in different substrates.

This was the second derivative of GS. The methodology produced systems with these properties as a side effect of correct practice. The BIOISOs made the properties visible, named them, and pointed toward the next question: if a system already has most of what makes a biological organism self-maintaining, what would it take to close the remaining gap?

That question is what became the fifth tier.

The Cascade

Once the methodology was stable enough to name, the structure that had emerged from the experiments became visible as a cascade — seven categories of practitioner obligation, each one removable once the specification at the layer below was complete.

You do not write code. The specification — architectural constitution, structural files, behavioral contracts — is precise enough that a stateless reader derives the implementation. The blueprint is the program.

You do not read the generated code. The behavioral harness verifies it automatically: API contracts, integration tests, mutation testing, quality gates. If verification fails, the specification is tightened and the derivation runs again. You do not audit the output. You define what correct looks like and let the harness confirm it.

You do not touch infrastructure. The same specification that governs the code governs the deployment environment: CI/CD pipelines, environment configurations, compliance gates. A CLI command is a specification gap. State the desired environment; the AI resolves it.

You do not diagnose bugs. Runtime signals are evaluated against the same formal properties that drove construction. Drift from specification is a specification violation, detectable and correctable by the same

mechanism that built the system. The Eye watches; the system corrects.

The system evolves itself. A GS-governed system already has the structural properties the BIOISOs identified. Close the remaining gap and the system can govern its own mutation — applying changes, verifying them against the telos, committing them if they pass, discarding them if they don't. The program evolves within the constraints of its specification. You do not maintain it. It maintains itself.

You do not design the system. State the problem. A colony of self-governing programs derives itself from that statement — each with its own telos, interacting through typed channels, expiring when their purpose is fulfilled. The architect's role dissolves into the problem-holder's role. This is Conclave.

The process observes itself. A system that has accumulated enough formal history of the practitioner's work infers what is needed before it is asked. This is the logical terminus of the cascade. It is stated here as a research question, not an implementation claim, because governance must precede capability. What T7 requires is not more formal theory. It is a careful answer to who decides what the practitioner needs before any autonomous inference becomes action.

Each tier rests on the one below it. Each tier is made possible by a specification precise enough that a reader carrying no prior context can derive correct behavior from it alone. All seven tiers are possible because AI systems understand human language at a level that turns specifications written for humans into programs executable by machines.

Why I Was Ready for This

I did not come to this question by accident.

My master's degree in data science at the Universitat Oberta de Catalunya ended with a thesis on natural language processing. I chose it because I had always been drawn to languages — the way meaning transfers between minds through symbols, the way a sentence in one language carries a structure that does not translate directly into another, the way context shifts meaning without changing a word. Word2vec interested me because it was the first time I had seen a machine represent meaning as geometry — words as vectors in space, relationships as directions, analogy as arithmetic. The machine did not know what the words meant in any human sense. But the structure of the relationships was there.

That thesis was about the question of whether machines could understand human language well enough to be useful. The answer in 2015 was: partially, in specific domains, with significant limitations.

The answer in 2025 was different. Large language models trained on the full written record of human knowledge read formal specifications the way a senior engineer reads them — not by parsing syntax trees but by recognizing intent, inferring constraints, understanding what the architect was trying to say. That shift is what makes Generative Specification possible. The specification is written in human language because it is meant for a human-level reader. The AI is that reader now.

The thread that runs from the word2vec thesis to the seven-tier cascade is the same thread: the question of what becomes possible when a machine can understand what you mean.

It turns out the answer is most of software engineering.

Juan Carlos Ghiringhelli April 2026